B.Sc. (Hons) in IT – Field of Specialization Interactive Media
2026 – Year 04 – Semester 02

SE4041 – Mobile Application Design and Development

Assignment 02

Final Report

PaperSprint

Learn a little. Go a little further.

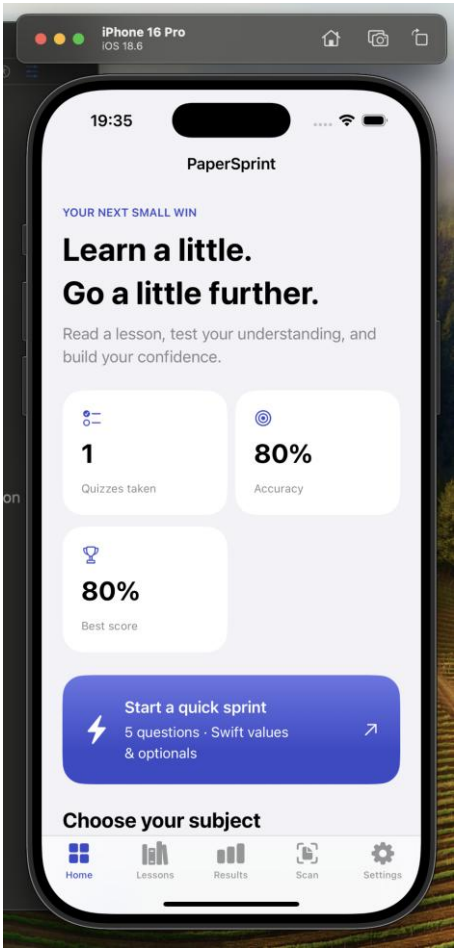
Student Name: Himsara MKDL

Student ID: IT22274298

GitHub Repo.: https://github.com/HimsaraMKDL/PaperSprint-iOS

Short lessons. Better revision.

PaperSprint combines guided study and low-friction assessment in one native iOS experience.



PROBLEM AND AIM

Learners often move between static notes and separate quiz resources. PaperSprint brings concise lessons, immediate answer checking and progress history into a single focused workflow.

PRIMARY AUDIENCE

Undergraduate students and beginner programmers revising Swift, SwiftUI and general iOS-development concepts.

CORE PROPOSITION

STUDY Structured lessons with original examples and explanations.

PRACTISE Five multiple-choice questions for each lesson.

REVIEW Save scores, review answers and monitor progress.

SCAN Extract printed text through Apple Vision OCR.

CAPTURED DEMONSTRATION

The dashboard screenshot shows one completed attempt, 80% answer accuracy and an 80% best score. These figures reflect the photographed sample state, not a multi-user evaluation.

Browse. Select. Understand.

A compact, five-tab layout gives students direct access to study content and results.

Navigation uses Home, Lessons, Results, Scan and Settings. Subject cards open relevant lessons; the Lessons tab also exposes the complete revision catalogue. Students can find a topic and begin a focused learning session without requiring an account.

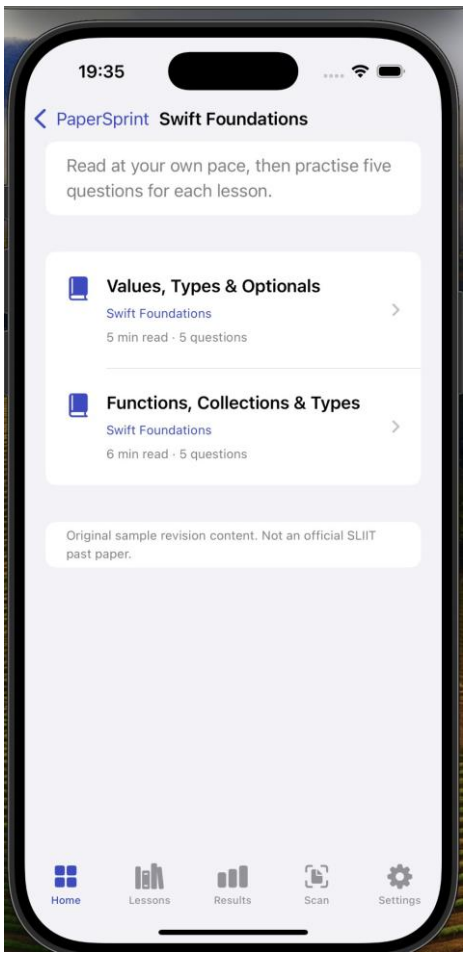


FIG. 02 Swift Foundations subject and two lessons

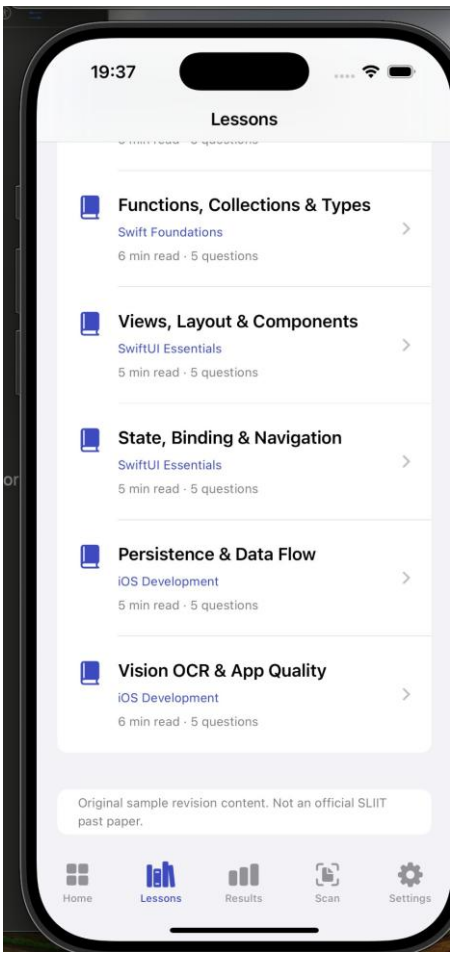
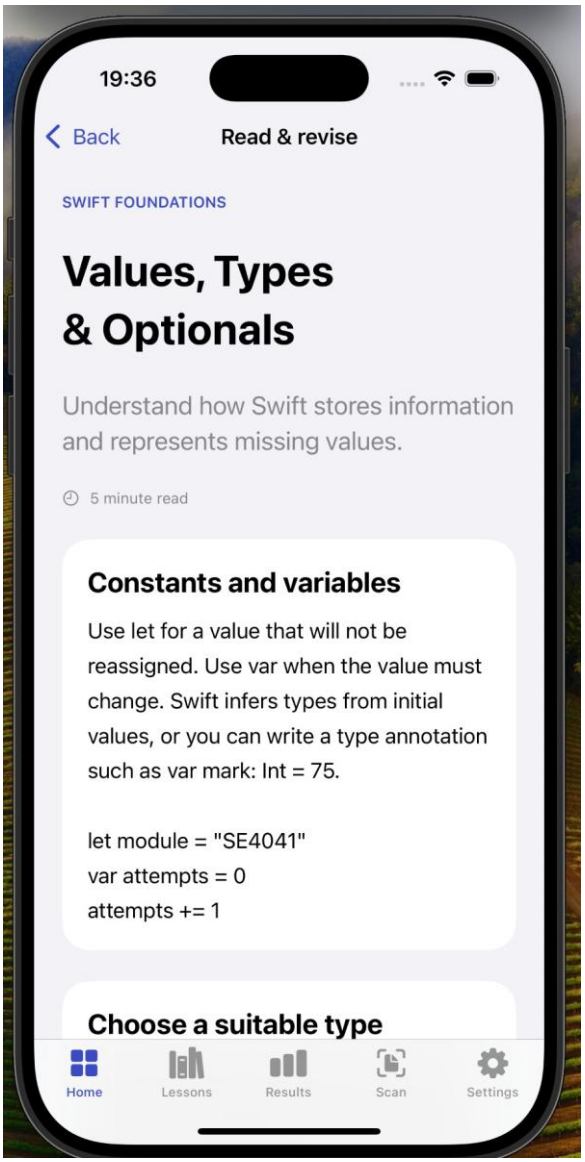


FIG. 03 All lessons grouped across three subjects

Reading before testing.

Readable lesson cards connect key concepts to concrete Swift examples.



LESSON STRUCTURE

A selected lesson opens a dedicated reading screen with a topic label, title, short description, estimated reading time and content sections. The example shown explains constants, variables and type inference.

TEACHING APPROACH

CHUNKED CONTENT Information is divided into concise sections to support short study sessions.

PRACTICAL EXAMPLES Swift syntax appears inside topic explanations, supporting conceptual recall.

QUICK PROGRESSION Each lesson connects to a five-question practice quiz.

SWIFTUI EXPERIENCE

The implementation uses NavigationStack, scrollable reading areas, system typography, rounded content cards and reusable interface components. The persistent tab bar keeps other app destinations accessible.

USER GUIDE

Open Home or Lessons → choose a subject and a lesson → read each content section → select “Practise this lesson” to begin assessment.

Each question becomes feedback.

Quiz scoring is deterministic, and completed attempts are stored for later review.

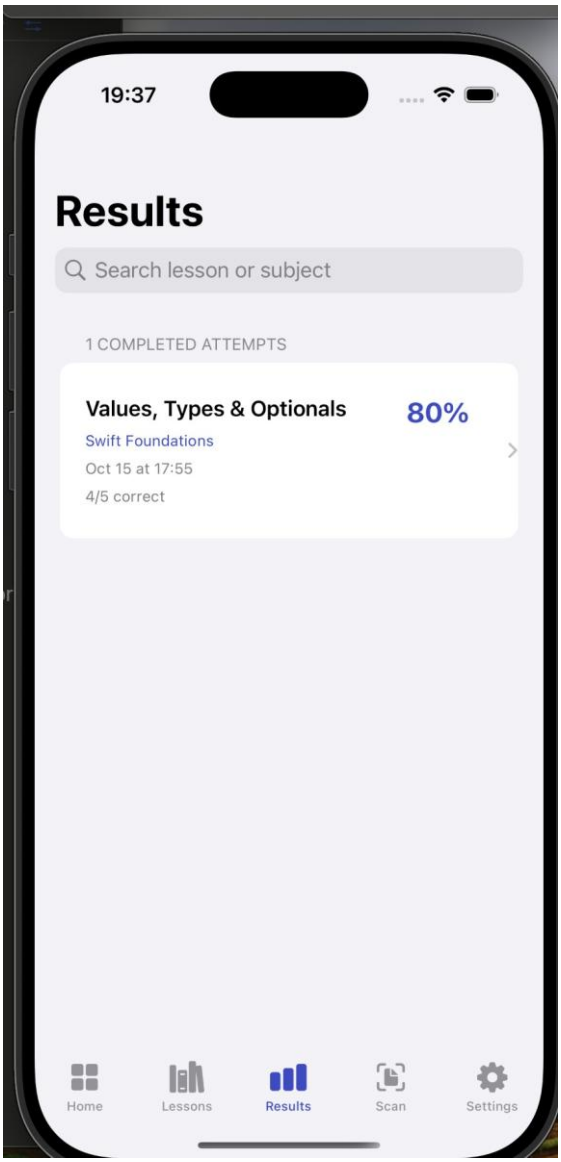


FIG. 05 Searchable results: 4/5 correct, 80% score

QUIZ PROGRESSION

A lesson offers exactly five multiple-choice questions. The student selects one option, taps “Check Answer”, reads correct/incorrect feedback with an explanation and advances to the next question.

RELIABILITY CONTROLS

ONE CHECK The checked option cannot be changed or counted twice.

REVIEW Saved answer snapshots retain the selected and correct options with explanations.

RETRY A new attempt begins with fresh question state.

SCORING

Score (%) = (number of correct answers ÷ total questions) × 100

In the screenshot, 4 correct answers out of 5 produce an 80% score. The result card displays its lesson, subject, timestamp and count.

PERSISTENT STORAGE

SwiftData saves a QuizResult containing a unique attempt ID, date, lesson/subject identifiers, totals, percentage, duration and encoded answer records. Finished attempts survive ordinary app relaunch; unfinished attempts are not saved.

The Results tab includes history search and deletion controls. Persistence uses a local ModelContainer rather than a remote database, with an explicit save/retry path if storage fails.

From paper to selectable text.

A real on-device Apple Vision OCR workflow supports image-based study material.

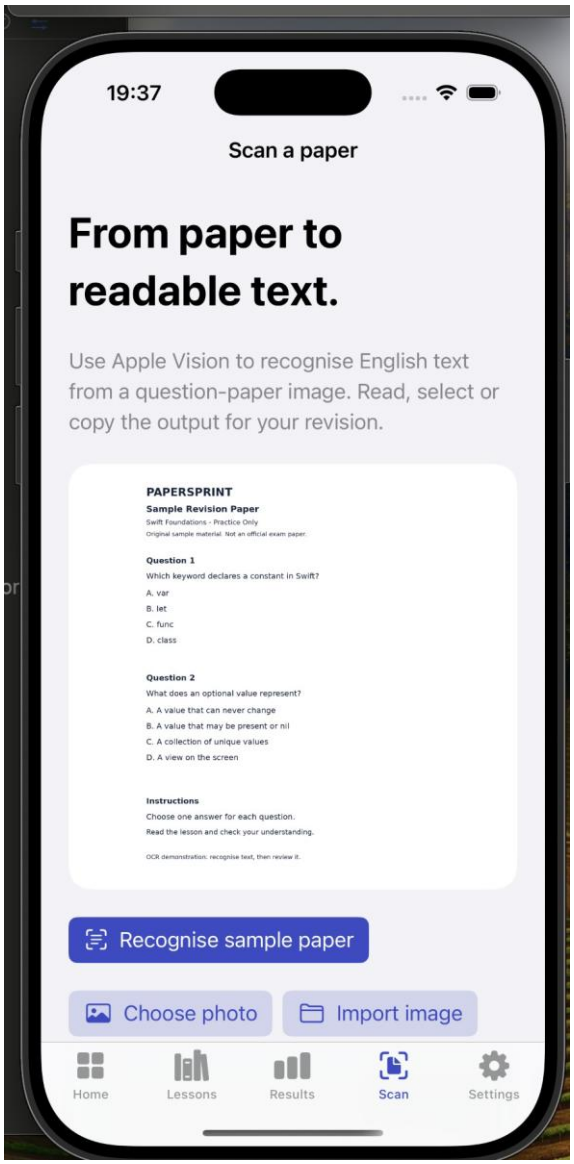


FIG. 06 Sample revision paper and OCR input options

FRAMEWORK INTEGRATION

Apple Vision processes revision-paper images using VNRecognizeTextRequest and VNImageRequestHandler. Recognition produces selectable English text rather than relying on a manually typed or pre-stored answer.

INPUT PATHS

SAMPLE Recognise an original sample paper bundled with the app.

PHOTOS Choose an existing image through PhotosUI.

FILES Import a supported decodable image from Files.

IMPLEMENTATION DETAILS

OCRWorker is an actor that handles image decoding and Vision recognition outside the main UI actor. ImageIO applies image orientation and downsamples large images. The interface provides loading feedback, copy and share actions.

DEFINED FEATURE BOUNDARY

This is text extraction, not an automatic quiz generator or answer grader. The feature accepts images rather than PDFs, and recognition accuracy varies with image quality. The imported text is not stored as quiz content.

The in-app sample image is explicitly labelled original practice material; no official past-paper ownership is implied.

Personalise without complexity.

System-consistent controls and readable states support a focused student experience.

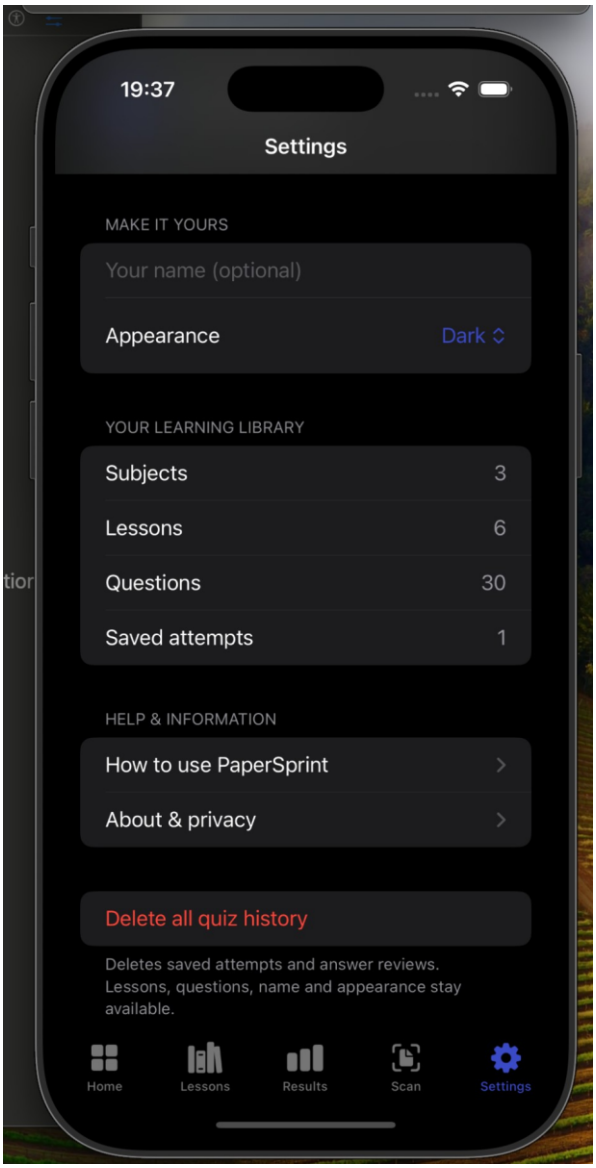


FIG. 07 Dark Mode, library information and history controls

APPEARANCE AND SETTINGS

Students can optionally enter a display name and choose System, Light or Dark appearance. The captured screen illustrates the native Dark Mode treatment of settings cards, labels and destructive actions.

ACCESSIBILITY CHOICES

SYSTEM TYPE Semantic text styles are used for scalable reading.

MORE THAN COLOUR Answer states use readable text as well as visual styling.

ADAPTIVE MOTION Animations consider Reduce Motion preferences.

INTUITIVE LAYOUT Five bottom tabs and familiar iOS controls reduce navigation effort.

PRIVACY AND OWNERSHIP

Quiz history is saved locally in the app sandbox. “Delete all quiz history” requires confirmation and removes saved attempts without removing the bundled lessons, user name or appearance preferences.

CAPTURED LIBRARY STATE

The Settings view reports 3 subjects, 6 lessons, 30 questions and 1 saved attempt in this demonstration.

Design note: accessibility support is present in the source, while complete VoiceOver, large-text and orientation evaluation should be documented separately if performed.

Clear roles. Maintainable logic.

PaperSprint separates local content, navigation, quiz rules, persistence and OCR processing.

ARCHITECTURE OVERVIEW

PRESENTATION	DOMAIN	PLATFORM
SwiftUI Views Home / Lessons / Quiz / Results / Scan / Settings	Models + QuizService Question selection / score rules / answer records	SwiftData + Vision Persistent results / on-device OCR

The Views coordinate the user journey; quiz state is managed by QuizService; SwiftData models store completed attempts; the OCRWorker actor handles recognition. UI components are reused instead of implementing each card independently.

SOURCE MAP AND RESPONSIBILITIES

Project location	Function
PaperSprintApp.swift	ModelContainer configuration, app launch and appearance
ContentView.swift	TabView and nested NavigationStack routing
Models / LearningModels.swift	Subject, Lesson, Question, AnswerRecord, QuizResult
Resources / SampleLessons.swift	Three subjects, six lessons, thirty multiple-choice questions
Services / QuizService.swift	Answer checking, transitions, score calculation
Services / OCRWorker.swift	Image decoding, Apple Vision text recognition
Components / LearningComponents.swift	SubjectCard, AnswerOptionView, ScoreRing, PaperCard
Views / *.swift	Study screens, saved results, scanner and settings

TECHNICAL BOUNDARIES

Minimum deployment: iOS 17.0. The iPhone 16 Pro simulator shown runs iOS 18.6. The local lessons, quiz state and bundled OCR example require neither an account nor an external service. SwiftData history is stored locally, while app preferences are stored using AppStorage.

ENGINEERING PRACTICES

Strongly typed models, reusable views, a state-based quiz service, unique attempt identifiers, explicit persistence handling and actor-isolated OCR processing contribute to readability, testability and predictable interaction.

From build to submission.

The report records evidence available from the supplied project and simulator captures.

TESTING AND DEMONSTRATED EVIDENCE

Area	Observed / source-supported evidence	Evidence status
App and navigation	Seven iPhone 16 Pro simulator screenshots illustrate core screens	Captured / user-tested
Assessment outcome	Saved quiz attempt displays 4/5 correct = 80%	Captured
Persistence	Result record visible in Results; SwiftData model and save code included	Visible; relaunch not pictured
Apple Vision OCR	Scan interface with sample and import controls; Vision implementation exists	UI captured; output not pictured
Automated testing	Swift Testing and XCTest test sources included in ZIP	Execution log not supplied
Usability and accessibility	Dark appearance and navigation demonstrated; other checks documented in source	Dark Mode captured

QUICK USER GUIDE

1 Launch PaperSprint → tap Home or Lessons → choose a subject and open a lesson.

2 Read lesson sections → tap “Practise this lesson” → complete five questions.

3 Select each answer, check feedback, finish and open Results to review saved attempts.

4 Open Scan for sample/Photos/Files OCR → open Settings for appearance or history control.

CHALLENGES, CHOICES AND LIMITATIONS

SwiftData initialisation and SwiftUI navigation-API compatibility required build corrections during development. Quiz state guards address double-scoring, while OCR preprocessing handles diverse source images. The current catalogue is intentionally small and local, and OCR does not generate new quiz questions.

AI-ASSISTED DEVELOPMENT AND REFERENCES

ChatGPT/Codex assisted with planning, SwiftUI structure, implementation review, debugging and documentation. AI-generated suggestions were incorporated into local Xcode work. The assignment also requests the underlying prompt-and-response record; a conversation export can be submitted separately.

Sources: Apple Developer Documentation — SwiftUI, SwiftData, Vision, PhotosUI and XCTest; SE4041 Assignment 02 brief (2026).

CONCLUSION

PaperSprint demonstrates a cohesive native revision workflow, reusable SwiftUI presentation, persistent quiz history and a meaningful Apple Vision integration. The user reports successful simulator operation; any formal automated-test or participant-study claims should be accompanied by their own execution evidence.